

Wildcards in Generics

Date:

YOUVA
STELLAR

Generics

→ General purpose class

Object $\longleftrightarrow$ class `Box<T> {`
T values
`}`

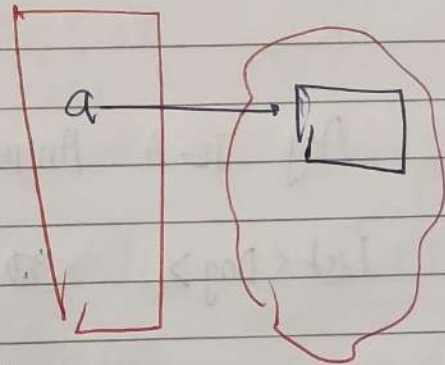
Generics → (Break → Parent-child Relation)

`eat()`
`walk()` `Animal`

`bark()` `Dog`

`Animal a = new Dog();`

`a.eat()`
`a.walk()`
`a.bark()` ✗

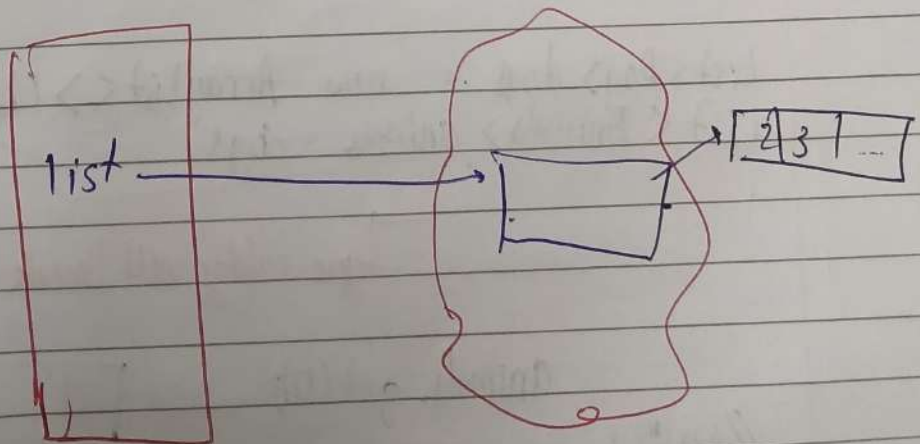


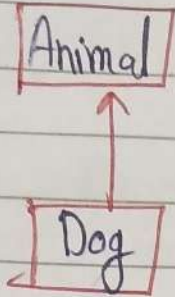
`List`

`ArrayList`

`List<Integer> list = new ArrayList<>();`

`list.add(2)`
`list.add(3)`

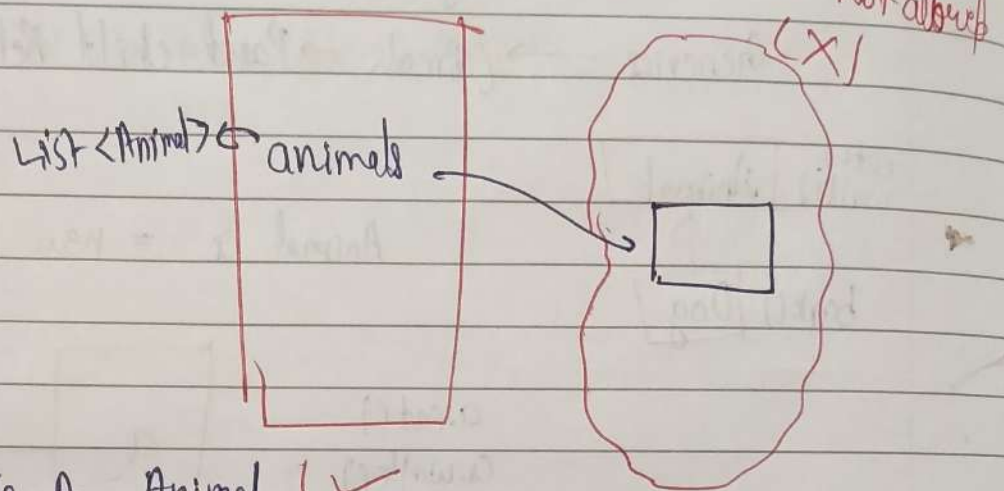




Animal a = new Dog();

List<Dog> dogs = new ArrayList<>();

List<Animal> animals = dogs;



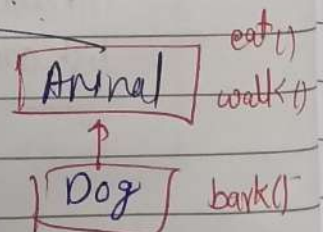
Dog Is-A Animal ✓

List<Dog> ~~Is-A~~ List<Animal> (X)
 ↓
 Not any relation

Generic <A> is NOT subtype of Generic

List<Dog> dogs = new ArrayList<>();
 List<Animal> animals = dogs

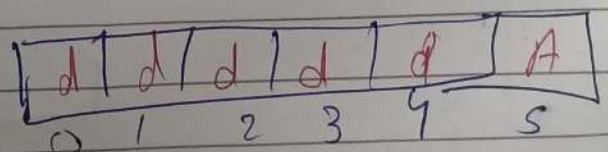
type safety will break



animals.get(0);

//writing

animals.add(new Animal());




```
for (Dog d: animals) {
    d.bark();
}
```

// but how will Animal run bark()
 this becomes error so java doesn't
 allow this

Generics are invariant

→ if A is child of B
 Generic <A> is NOT child of Generic

But in Arrays parent child relationship is valid

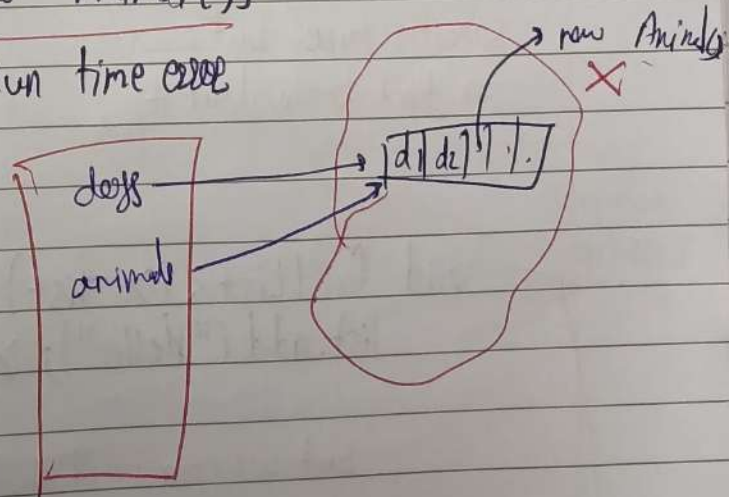
→ are covariant but risky

```
Dog[] dog = new Dog[10];
Animal[] animals = dogs; (✓)
```

animals[2] = new Animal();

→ Run time error

Dog d = new Animal();
 (X)



void printAnimals (List<Animal> a) {
 for (a _____) {
 }
}

how can pass any List

List<Dog> dogs = new ArrayList<>();
 printAnimals (dogs);
 compiler error

due to this problem java introduced "Wildcards."

* Wildcards (?)

List<?> list = new ArrayList<Animal>();
 list of
 specific type but
 we don't know about it
 Dog
 Cat
 ...

void fun (List<?> list) {
 list.add("Hello"); X not possible because compiler doesn't
 know type of ~~obj~~ list is there
 but we can read it. } but we don't know in which data
 type we can store. except
 Object class

Object obj = list.get(0);

}

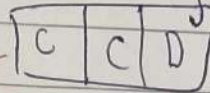
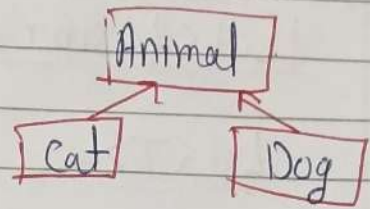
so ~~but~~ Wildcards provide very limited
 scope

but
 upper bounds could
 also be used to broaden
 the scope.


```
fun(List <? extends Animal> l) {
    1. add(new Dog());
    1. ad(new Animal());
}
```

Animal or its subtypes

```
fun(cats);
fun(Dog);
```



wrong that is why adding is prohibited but reading is allowed

Covariant

super



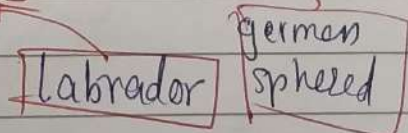
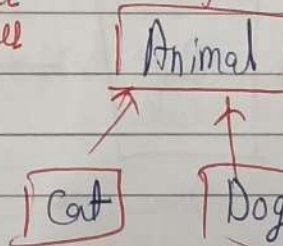
Functions with lower bound

```
List<? super Animal>
```

Animal or its super types

Object

```
fun(List<? super Animal> list) {
}
```



```

list fun(List<Animal>) ✓
fun(List<Object>) ✓
fun(List<Dog>) ✗
fun(List<Cat>) ✗

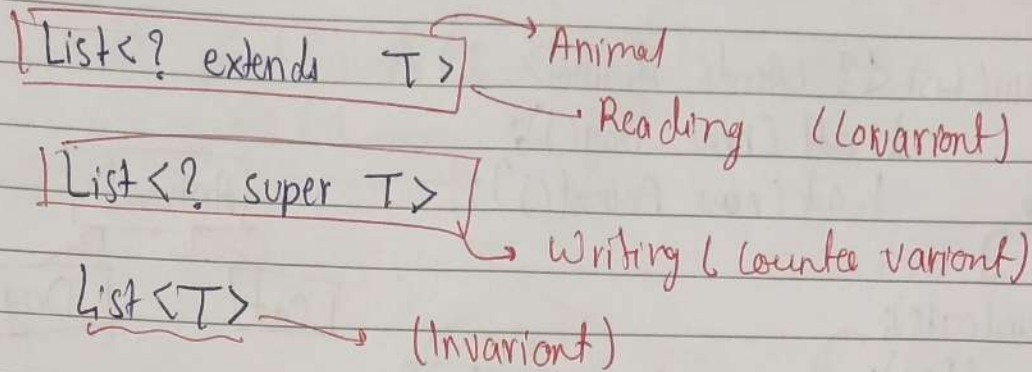
```

```
fun(List<? super Animal> l) {
    1. add(new Animal()); // now possible
    1. add(new Labrador());
    1. add(new GermanShpherd());
    Animal a = 1. get(0) ✗
}
```

```
fun(Animal);
fun(Object);
```

Object obj = 1. get(0) we can read only in object class

but now we cannot read this as we do not know if it's Animal or Object & so on

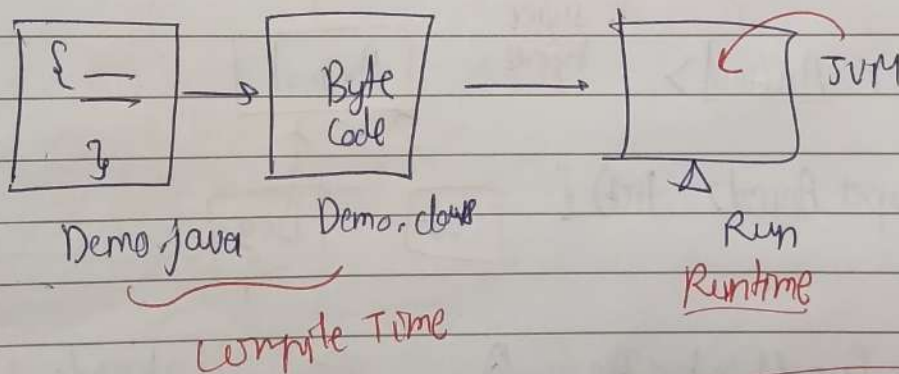


PECS Rule

→ producer extends
→ consumer super

* Type Erasers

Does JVM knows Generics? → No



```
class Box <T> {
    T value;
}
```



```
class Box {
    Object value;
}
```

Bytecode

```
class Pair <T, U> {
    T first;
    U second;
}
```



```
class Pair {
    Object first;
    Object second;
}
```


1) if No Bound → Replace with Object

`class Box <T> {
 T value;
}` → `class Box {
 Object value;
}`

2) if Bounded → replace with bound

`class Box <T extends Number> {
 T value;
}` → `class Box {
 Number value;
}`

3) Insert Casting automatically

`class Box <T> {
 T value;
}` → `class Box {
 Object value;
}`

`Box <Integer> b1
= new Box <> (5);
Integer x = b1.value();`

`Box b1 = new Box(5);
Integer x = b1.value()
(Integer)`

① ~~if (l instanceof List<String>) {~~ `List<String> l = new ArrayList<>();`

`if (l instanceof List<String>) {`

`}`

// not allowed

`List l = new ArrayList`

② No method overload (x)

`void print(List<String> l) { }`

`void print(List<Integer> l) { }`

`void print(List l) { }`

`void print(List l) { }`

③ Compiler Generated Bridge Method:

`class Parent<T> {`

`T get() {`

`return null;`

`}`

`}`

`class Parent {`

`Object get() {`

`return null;`

`}`

`}`

`class Child extends Parent<String> {`

`@Override`

`String get() {`

`return "Hello";`

`}`

`}`

`class Child extends Parent {`

`String get() {`

`return "Hello";`

`}`

`Object get() {`

`get();`

`}`

`}`

Bridge Method

① Type Erases?

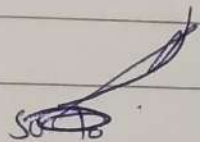
Generics → from Java 5

JVM
Compiler
safety test

before java 5 usually: List l = new ArrayList();

From Java 5

usually List<Integer> l = new ArrayList<>();



so not to break before Java 4 ecosystem which doesn't follow generics. java introduced type eraser

② Generics → Not supported primitives. Why?

```
class Box <T> {
```

```
    T value;
```

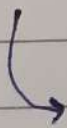
```
}
```

Object → but int is not a class

so compiler cannot convert T with object

```
Box<int> b1 = new Box<>();
```

in case of int (or other primitives)



```
Box b1 = new Box();
```